

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset. Also code the perceptron learning algorithm for OR, NOT, and AND gates using Python. Show the weights after each update. Plot the decision boundary after each weight update

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Experimental Results

The Banknote Authentication Dataset was successfully loaded and analyzed. The dataset consists of 1372 samples with four numerical features, namely Variance, Skewness, Kurtosis and Entropy. No missing values were observed in the dataset. The class distribution contains 762 authentic banknotes and 610 forged banknotes.

Table 1 presents the basic characteristics of the dataset used in this experiment. The dataset is reasonably balanced and does not contain any missing values, making it suitable for binary classification.

From Table 2, it can be observed that the four numerical features exhibit different ranges and distributions. Since the features are measured on different scales, feature normalization was

Table 1: Dataset Summary

Parameter	Value
Total Samples	1372
Input Features	4
Target Classes	2
Missing Values	0
Authentic Banknotes (Class 0)	762
Forged Banknotes (Class 1)	610

Table 2: Descriptive Statistics of the Dataset

Statistic	Variance	Skewness	Curtosis	Entropy	Class
Count	1372	1372	1372	1372	1372
Mean	0.4337	1.9224	1.3976	-1.1917	0.4446
Standard Deviation	2.8428	5.8690	4.3100	2.1010	0.4971
Minimum	-7.0421	-13.7731	-5.2861	-8.5482	0
25% Quartile	-1.7730	-1.7082	-1.5750	-2.4134	0
Median (50%)	0.4962	2.3197	0.6166	-0.5866	0
75% Quartile	2.8215	6.8146	3.1792	0.3948	1
Maximum	6.8248	12.9516	17.9274	2.4495	1

performed prior to training. No missing values were present, therefore no additional data cleaning was required.

The dataset was normalized using Manual Z-Score Normalization according to

$$X_{norm} = \frac{X - \mu}{\sigma}$$

where μ represents the mean and σ represents the standard deviation of the corresponding feature.

After normalization, the dataset was divided into training and testing subsets using an 80:20 split ratio. This resulted in 1097 training samples and 275 testing samples.

A Single Layer Perceptron was implemented from scratch and trained using the perceptron learning algorithm. The model was trained for 50 epochs with a learning rate of 0.01. During training, the number of misclassified samples gradually reduced over successive epochs, indicating successful convergence of the learning process.

The final learned weight vector obtained after training was

$$\mathbf{W} = [-0.1991, -0.2233, -0.2172, -0.0140]$$

and the corresponding bias value was

$$b = -0.1$$

The trained model was evaluated on the testing dataset using Accuracy, Precision, Recall and F1-Score.

Table 3: Classification Performance Metrics

Metric	Value
Accuracy	98.55%
Precision	99.17%
Recall	97.54%
F1-Score	98.35%

The confusion matrix obtained during testing is shown below.

Table 4: Confusion Matrix

Actual Class	Predicted Class	
	Authentic	Forged
Authentic	152	1
Forged	3	119

The model achieved an accuracy of 98.55% on the testing dataset. Out of 275 testing samples, only four samples were misclassified. The obtained precision, recall and F1-score values further indicate that the perceptron was able to distinguish authentic and forged banknotes effectively. The overall results demonstrate that the Banknote Authentication Dataset is largely linearly separable and can therefore be classified successfully using a Single Layer Perceptron.

7. Generated Plots with Interpretation

7.1 Feature Distribution Analysis

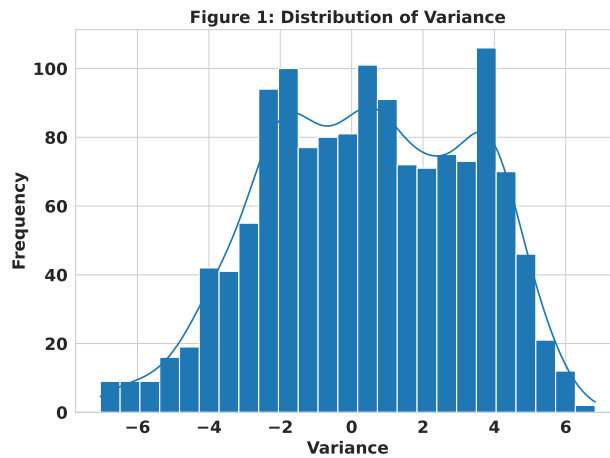


Figure 1: Distribution of Variance

Interpretation: The Variance feature spans a wide range of values from approximately -7 to +7. Most samples are concentrated around the central region, while fewer observations occur at the extreme ends. The broad spread of this feature suggests that Variance carries useful information for differentiating between authentic and forged banknotes.

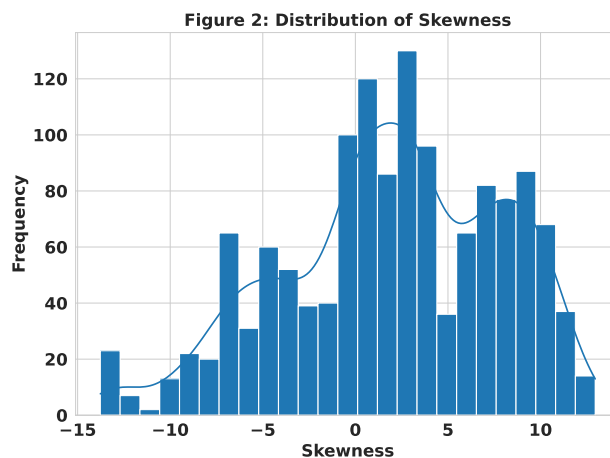


Figure 2: Distribution of Skewness

Interpretation: Compared to the other features, Skewness exhibits the largest spread of values, ranging from approximately -14 to +13. The distribution is not perfectly symmetric and contains observations across a wide interval. This indicates that the structural characteristics captured by skewness vary considerably among banknote samples.

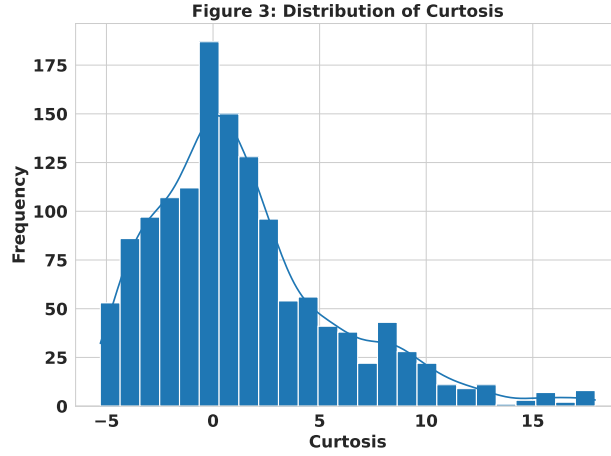


Figure 3: Distribution of Kurtosis

Interpretation: Kurtosis displays a noticeably wider positive range than the other features, extending up to nearly 18. A large number of samples are concentrated in the lower and middle regions, while a smaller number occupy the higher range. This variation suggests that Kurtosis contributes significantly to the overall discriminative power of the dataset.

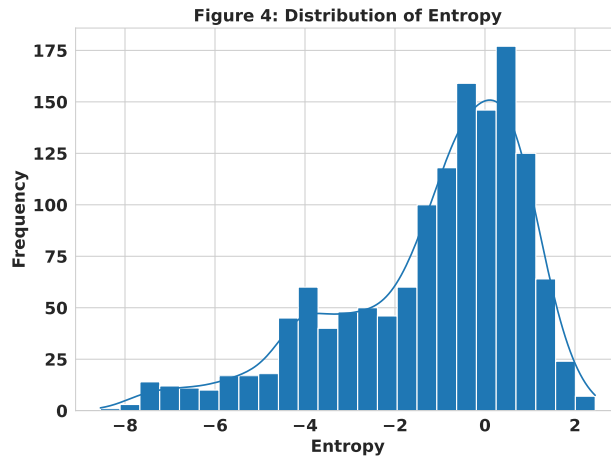


Figure 4: Distribution of Entropy

Interpretation: The Entropy feature is distributed over a comparatively narrower range than Variance, Skewness and Kurtosis. Most observations are clustered around negative and near-zero values. Despite its smaller range, the feature exhibits sufficient variation to assist the classifier in distinguishing between the two classes.

7.2 Feature Relationship Analysis

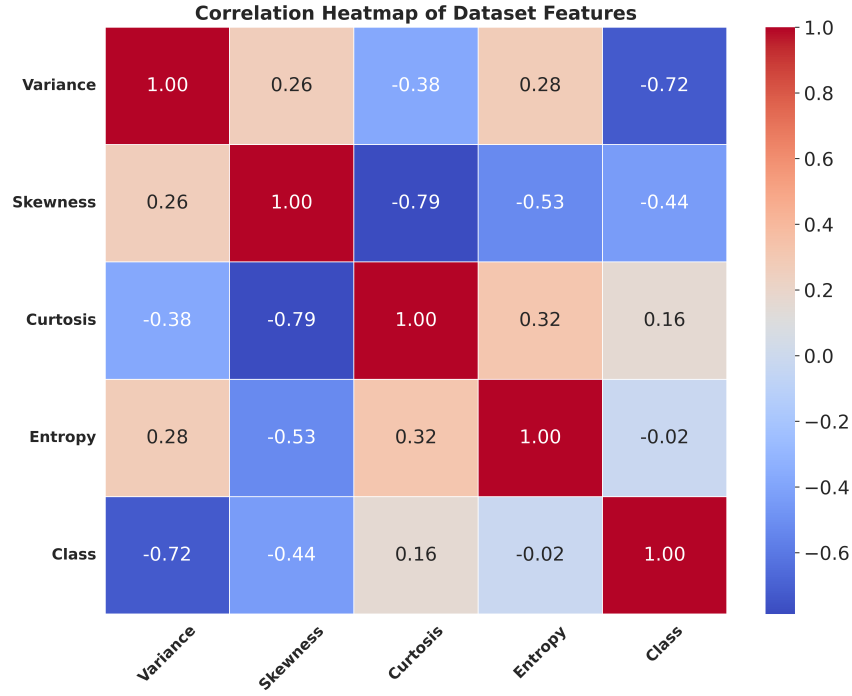


Figure 5: Correlation Heatmap of Dataset Features

Interpretation: The correlation heatmap reveals that some feature pairs exhibit moderate to strong relationships. Variance and Kurtosis show a noticeable positive correlation, whereas certain pairs involving Entropy exhibit weaker relationships. The absence of perfect correlations indicates that each feature contributes unique information to the classification process.

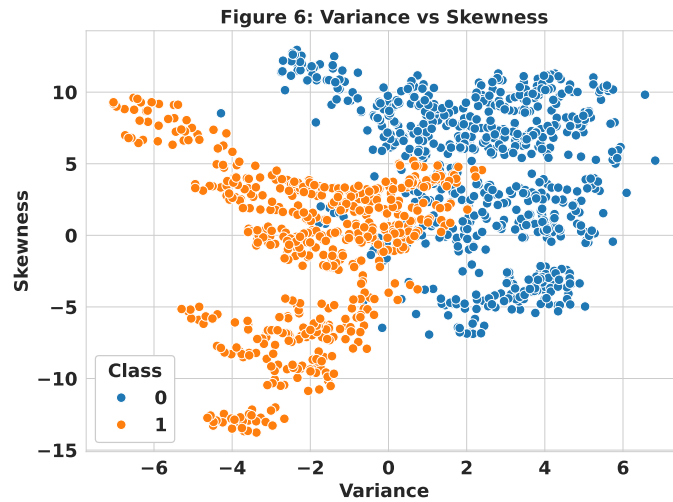


Figure 6: Variance versus Skewness

Interpretation: The scatter plot shows a visible grouping of samples when Variance and Skewness are considered together. Although some overlap exists between the two classes, distinct regions can still be observed. This indicates that these features jointly contribute to separating authentic and forged banknotes.

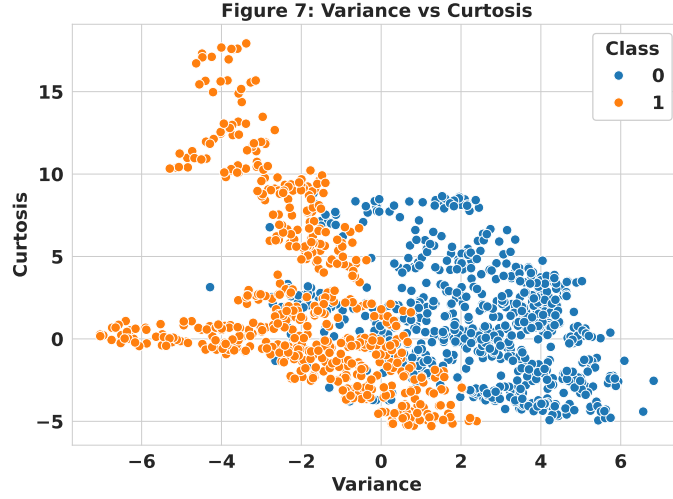


Figure 7: Variance versus Kurtosis

Interpretation: Among the scatter plots, the Variance–Kurtosis combination shows one of the clearest grouping patterns. A considerable number of forged banknotes appear concentrated in regions that differ from authentic banknotes. This indicates that Kurtosis complements Variance and improves the separability of the two classes.

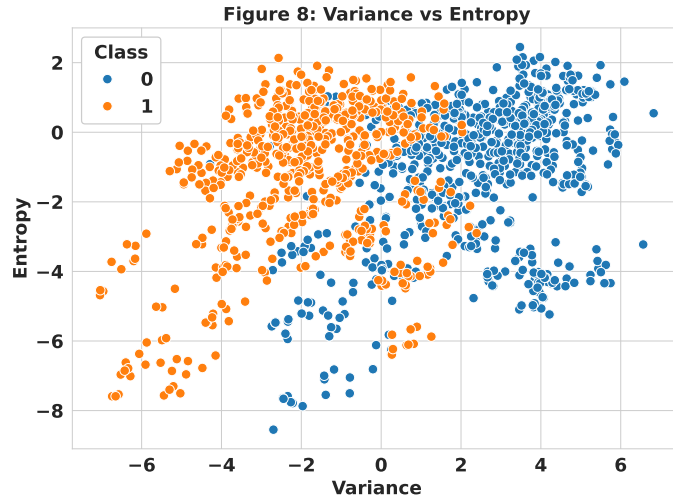


Figure 8: Variance versus Entropy

Interpretation: The Variance–Entropy feature space exhibits a more structured arrangement of samples compared to some of the other feature pairs. Although a small overlap between classes is visible, most samples tend to occupy different regions. This suggests that these two features contribute significantly towards defining the decision boundary.

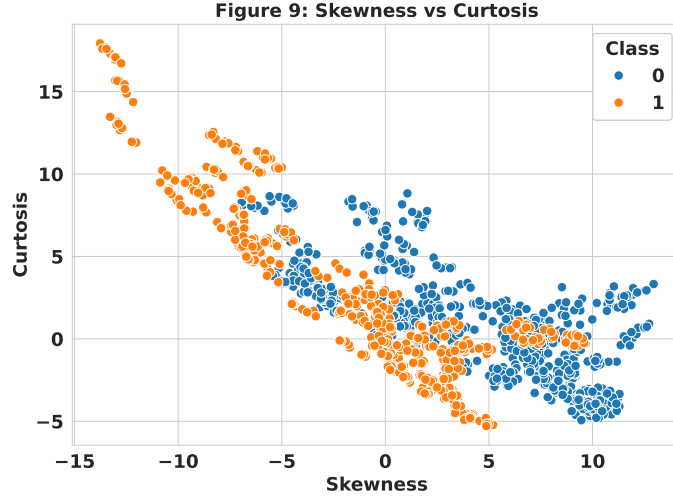


Figure 9: Skewness versus Kurtosis

Interpretation: The relationship between Skewness and Kurtosis reveals several dense clusters of samples. While some overlap is present, the overall distribution indicates that these statistical features capture meaningful differences between authentic and forged banknotes. Their combined behaviour contributes useful information for classification.

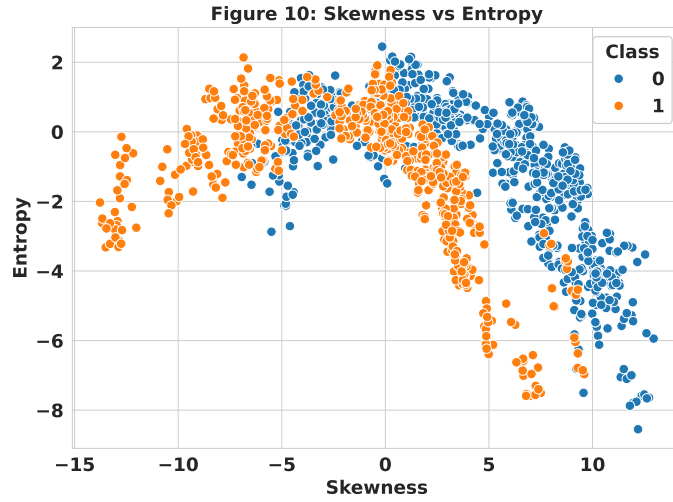


Figure 10: Skewness versus Entropy

Interpretation: The scatter plot demonstrates that the interaction between Skewness and Entropy produces identifiable regions within the dataset. Although the classes are not completely separable using only these two features, the observed grouping pattern suggests that both attributes contribute to distinguishing authentic and forged banknotes.

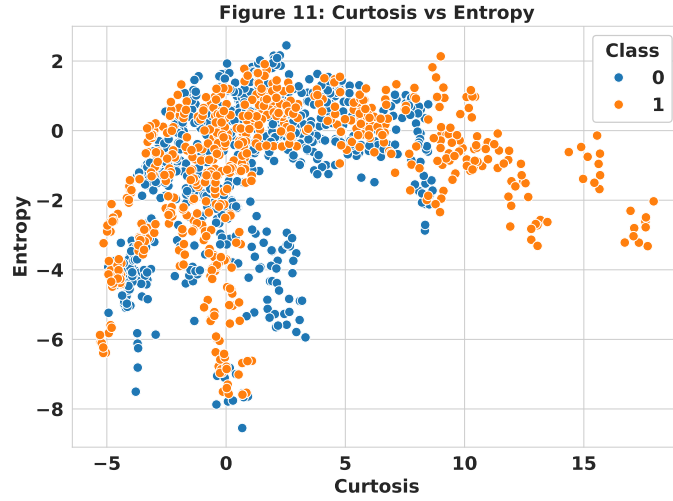


Figure 11: Curtosis versus Entropy

Interpretation: The Curtosis–Entropy feature space exhibits relatively clearer separation compared to several other feature combinations. Distinct concentrations of samples can be observed across different regions, indicating that these features collectively provide strong discriminatory information for classification.

7.3 Model Training and Evaluation

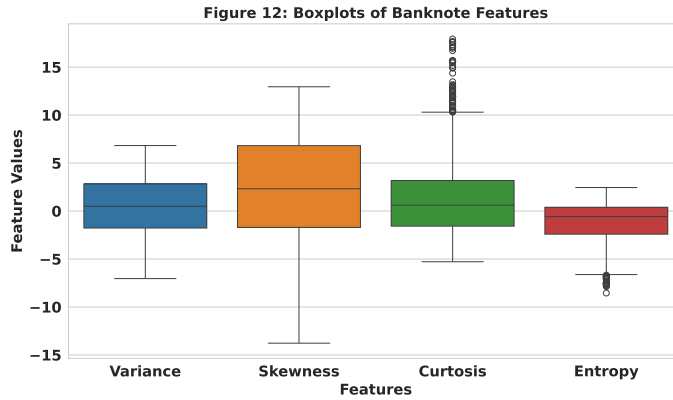


Figure 12: Boxplot of Dataset Features

Interpretation: The boxplot highlights the variation in scale among the four numerical features. Skewness and Curtosis exhibit larger spreads compared to Entropy, while a few extreme values can also be observed. This confirms the need for feature normalization before training, as features with larger magnitudes could otherwise dominate the learning process.

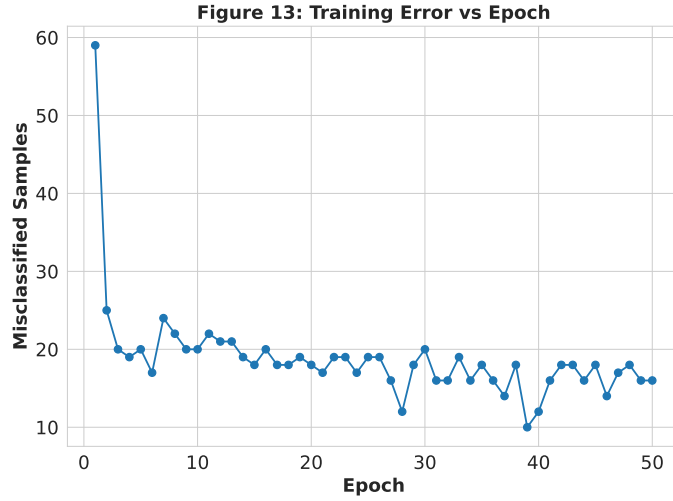


Figure 13: Training Error versus Epoch

Interpretation: The training error decreases progressively as the number of epochs increases. The perceptron initially misclassified a larger number of samples, but the error reduced steadily as the weights were updated using the learning rule. The flattening of the curve towards the later epochs indicates that the model has nearly converged and further training would result in only marginal improvements.



Figure 14: Evolution of Weights during Training

Interpretation: The weight trajectories show significant adjustments during the initial epochs, reflecting rapid learning from the training samples. As training progresses, the updates become smaller and the weight values stabilize. This behaviour indicates that the perceptron has identified a suitable set of parameters for separating the two classes.

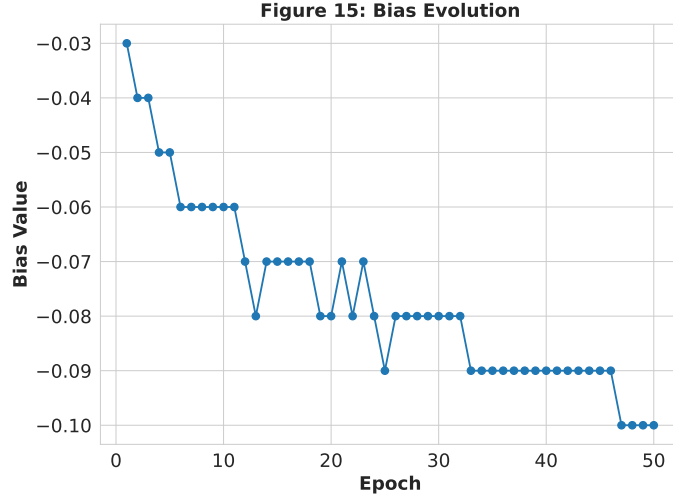


Figure 15: Bias Evolution during Training

Interpretation: The bias value changes throughout the training process and gradually settles near its final value. The stabilization of the bias indicates that the classifier has established an appropriate threshold for separating authentic and forged banknotes. The absence of large fluctuations during the later epochs further confirms convergence.

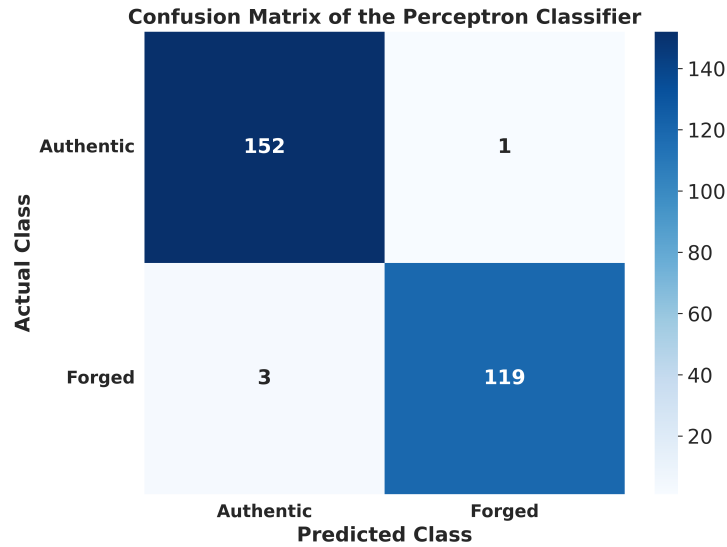


Figure 16: Confusion Matrix of the Perceptron Classifier

Interpretation: The confusion matrix shows that the classifier correctly identified almost all testing samples. Only a very small number of banknotes were assigned to the wrong class, resulting in an overall accuracy of 98.55%. The dominance of the diagonal entries indicates excellent classification performance and confirms that the perceptron learned an effective decision boundary.

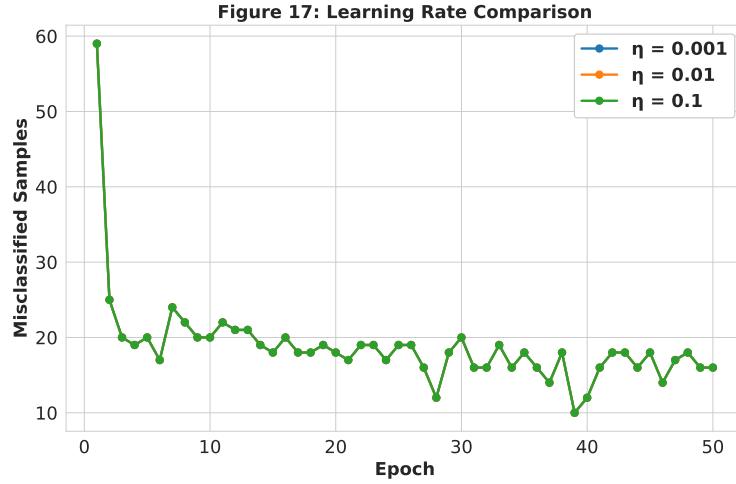


Figure 17: Comparison of Different Learning Rates

Interpretation: The learning rate comparison illustrates the influence of update magnitude on the convergence behaviour of the perceptron. Smaller learning rates produce gradual learning, whereas larger learning rates allow faster parameter updates. Despite the differences in convergence behaviour, all three learning rates achieved comparable classification performance, indicating that the dataset is sufficiently linearly separable and robust to moderate learning-rate variations.

8. Additional Perceptron Learning Experiments

To further understand the behaviour of the Perceptron Learning Algorithm, additional experiments were carried out using four basic Boolean logic gates: OR, AND, NOT and XOR. These logic gates provide simple datasets for studying the learning process of a single-layer perceptron. During training, the weights and bias were updated whenever a misclassification occurred, and the corresponding decision boundary was recorded after every weight update.

The objective of these experiments was to observe the convergence behaviour of the perceptron for linearly separable and non-linearly separable datasets. The complete training logs, truth tables and decision boundary plots were generated for each logic gate.

8.1 OR Gate

The OR gate outputs a logical value of 1 whenever at least one of the inputs is equal to 1. Since the output classes can be separated using a single straight line, the OR gate represents a linearly separable classification problem. The perceptron learning algorithm was trained using the OR truth table, and the weight updates together with the decision boundary evolution were recorded after every update.

8.1.1 Truth Table

Table 5: Truth Table for OR Gate

x_1	x_2	Output
0	0	0
0	1	1
1	0	1
1	1	1

8.1.2 Weight Updates

The perceptron required five weight updates before converging to the correct decision boundary. The weight and bias values after every update are presented in Table 6.

Table 6: Weight Updates during OR Gate Training

Update	Error	Weights (w_1, w_2)	Bias
1	-1	(0.0, 0.0)	-0.1
2	+1	(0.0, 0.1)	0.0
3	-1	(0.0, 0.1)	-0.1
4	+1	(0.1, 0.1)	0.0
5	-1	(0.1, 0.1)	-0.1

8.1.3 Decision Boundary Evolution

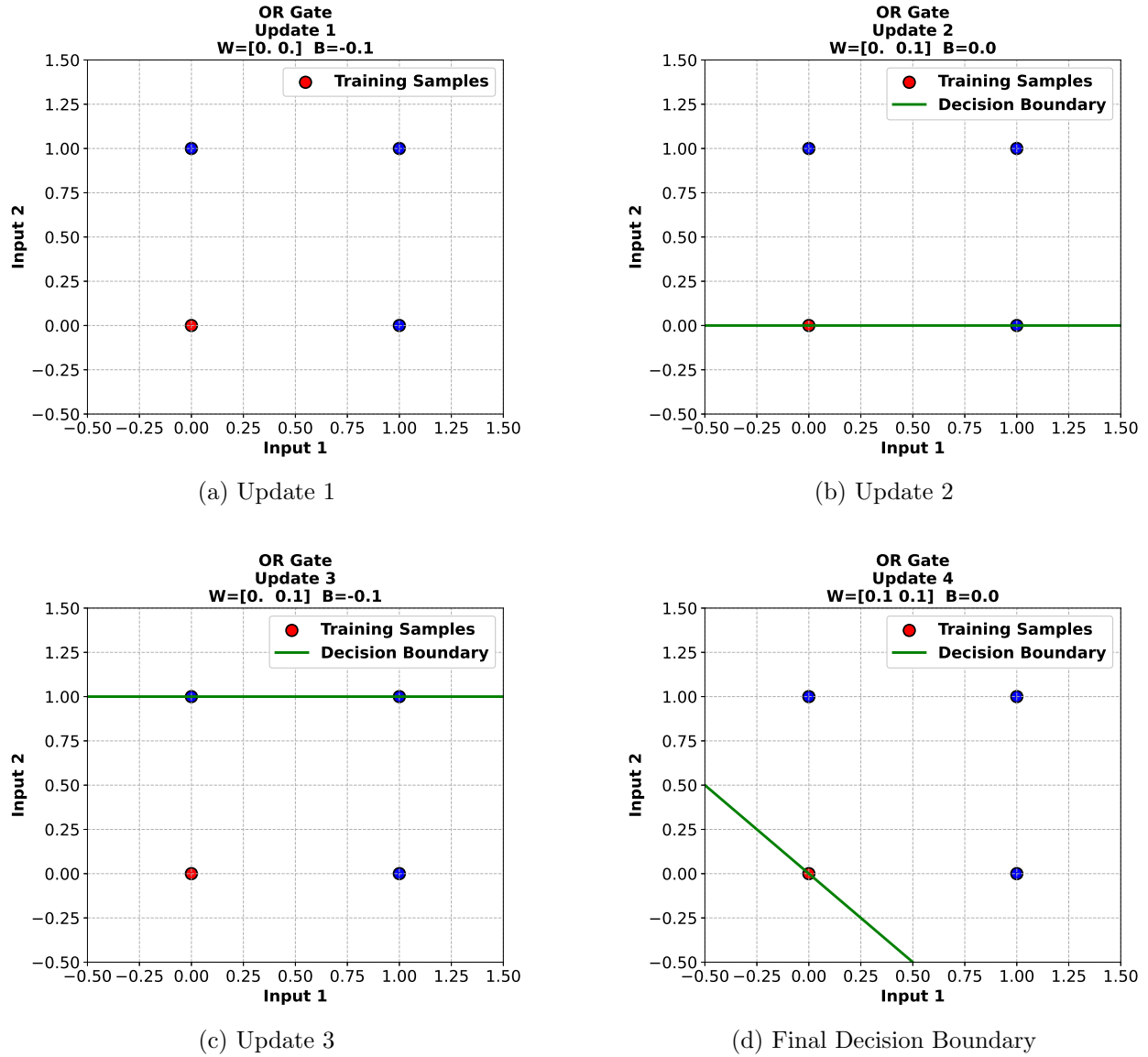


Figure 18: Evolution of the decision boundary during OR gate training.

8.1.4 Observation

The perceptron successfully learned the OR gate after five weight updates. During the initial iterations, the decision boundary shifted whenever a misclassified sample was encountered. After successive updates, the weights stabilized and a linear decision boundary was obtained that correctly classified all four input combinations. Since the OR gate is linearly separable, the perceptron converged without difficulty and achieved complete classification of the training data.

8.2 AND Gate

The AND gate produces an output of 1 only when both input values are equal to 1. Unlike the OR gate, the positive class is represented by a single input combination, making the learning process

slightly more challenging. However, since the AND gate is linearly separable, the perceptron learning algorithm is able to converge to a suitable decision boundary after a finite number of weight updates.

8.2.1 Truth Table

Table 7: Truth Table for AND Gate

x_1	x_2	Target	Predicted
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	1

8.2.2 Weight Updates

The perceptron required eight weight updates across three training epochs before converging. Table 8 summarizes the weight and bias values recorded after each update.

Table 8: Weight Updates during AND Gate Training

Update	Epoch	Error	Weights (w_1, w_2)	Bias
1	1	-1	(0.0, 0.0)	-0.1
2	1	+1	(0.1, 0.1)	0.0
3	2	-1	(0.1, 0.1)	-0.1
4	2	-1	(0.1, 0.0)	-0.2
5	2	+1	(0.2, 0.1)	-0.1
6	3	-1	(0.2, 0.0)	-0.2
7	3	-1	(0.1, 0.0)	-0.3
8	3	+1	(0.2, 0.1)	-0.2

8.2.3 Decision Boundary Evolution

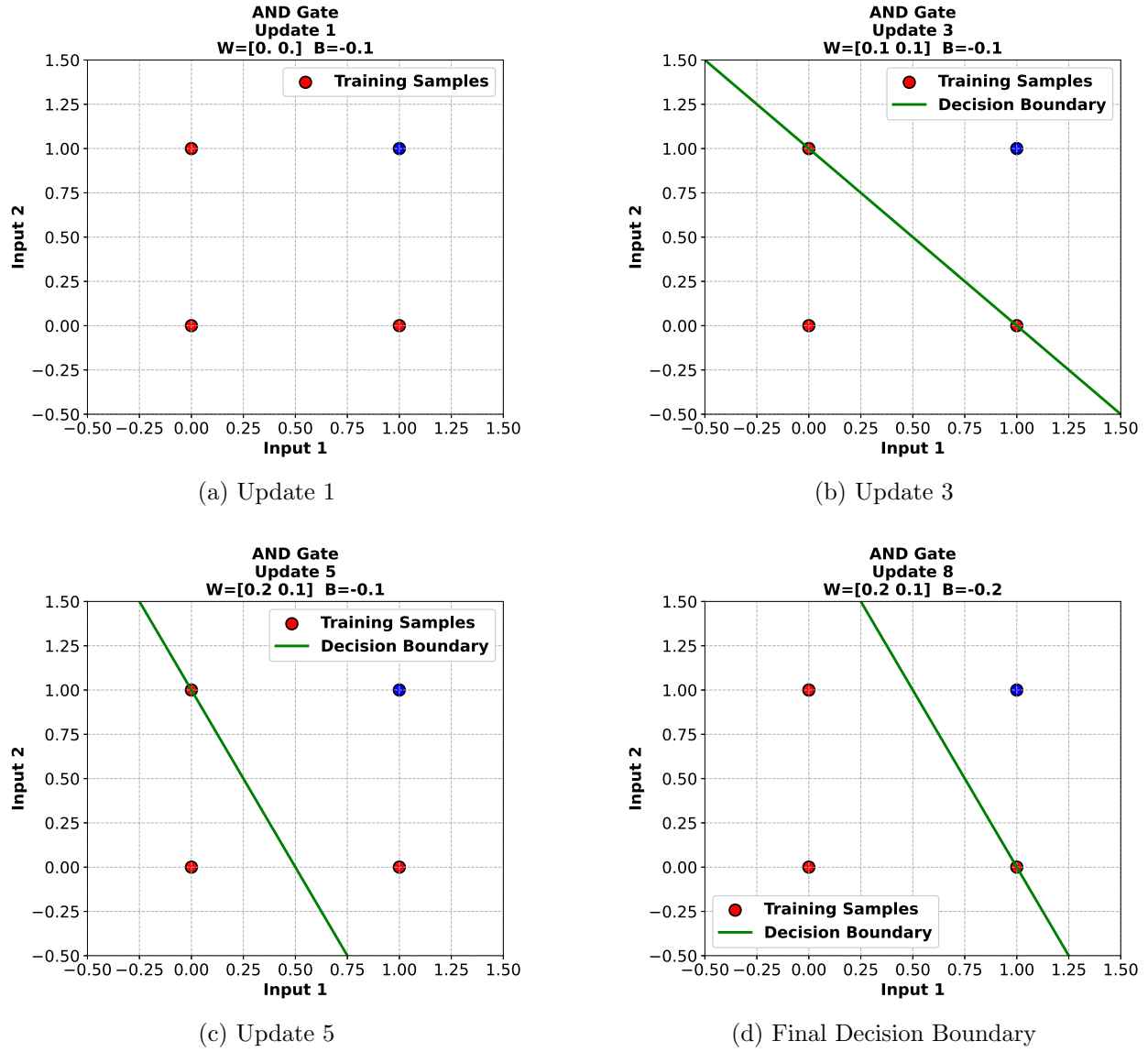


Figure 19: Evolution of the decision boundary during AND gate training.

8.2.4 Observation

The perceptron successfully learned the AND gate after eight weight updates distributed over three training epochs. During training, the weights and bias were adjusted whenever a sample was misclassified, causing the decision boundary to gradually move towards its optimal position. Unlike the OR gate, the AND gate required additional updates because only one input combination belongs to the positive class. Nevertheless, the final decision boundary correctly separated all four input combinations, demonstrating successful convergence of the perceptron on a linearly separable problem.

8.3 NOT Gate

The NOT gate performs logical negation by inverting the input value. An input of 0 produces an output of 1, while an input of 1 produces an output of 0. Since the two output classes can be separated by a single linear decision boundary, the perceptron learning algorithm is expected to converge rapidly with only a few parameter updates.

8.3.1 Truth Table

Table 9: Truth Table for NOT Gate

Input	Output
0	1
1	0

8.3.2 Weight Updates

The perceptron converged after only two weight updates. The corresponding changes in the model parameters are presented in Table 10.

Table 10: Weight Updates during NOT Gate Training

Update	Weights	Bias
1	(-0.1, 0.0)	-0.1
2	(-0.1, 0.0)	0.0

8.3.3 Decision Boundary Evolution

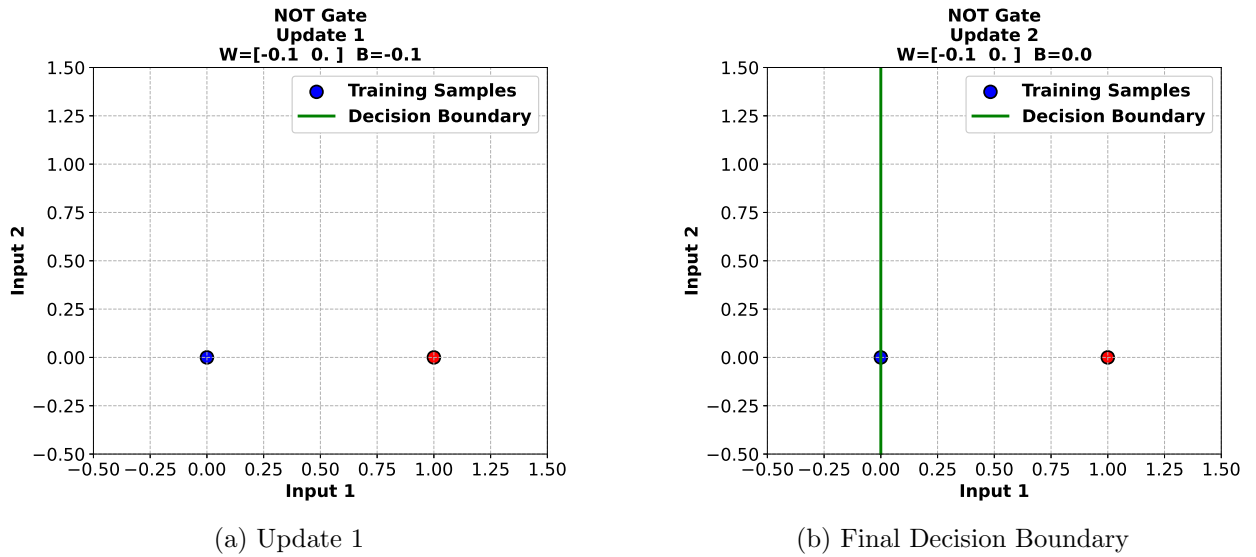


Figure 20: Evolution of the decision boundary during NOT gate training.

8.3.4 Observation

The NOT gate converged after only two weight updates, making it the fastest among the linearly separable logic gates considered in this experiment. The weight vector remained unchanged after the initial adjustment, while the bias was updated to establish the correct decision threshold. The final model correctly classified both possible input values, confirming that the perceptron efficiently learns simple linearly separable binary functions.

8.4 XOR Gate

The Exclusive OR (XOR) gate outputs a logical value of 1 only when the two input values are different. Unlike the OR, AND and NOT gates, the XOR gate is not linearly separable. Consequently, no single straight line can divide the input space into the required output classes. Therefore, a Single Layer Perceptron cannot learn the XOR function, irrespective of the number of training epochs or weight updates.

8.4.1 Truth Table

Table 11: Truth Table for XOR Gate

x_1	x_2	Output
0	0	0
0	1	1
1	0	1
1	1	0

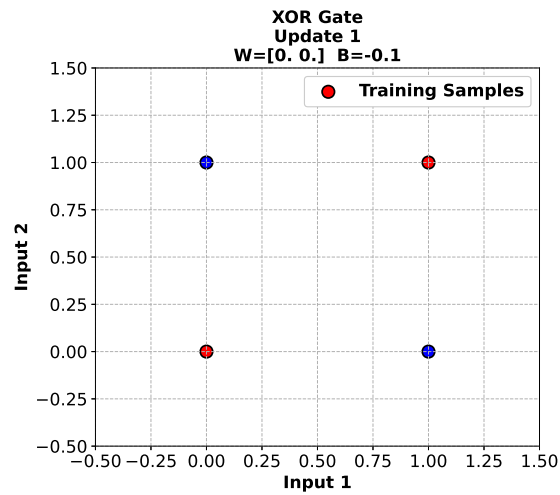
8.4.2 Training Summary

Unlike the previous logic gates, the perceptron was unable to converge while learning the XOR function. During training, the weights continued to change as the algorithm repeatedly attempted to correct previously misclassified samples. Since no linear decision boundary exists for the XOR problem, convergence was never achieved.

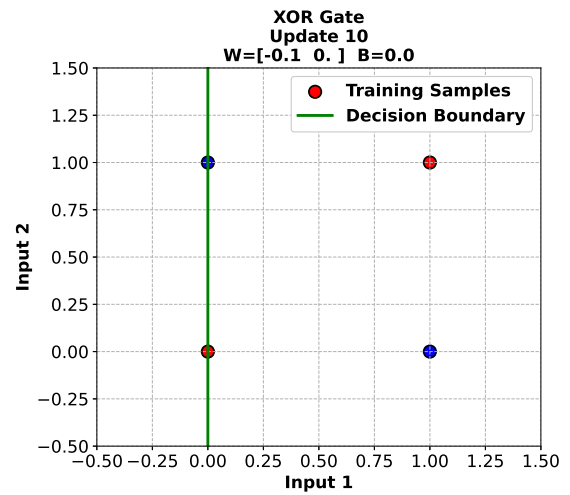
Table 12: Training Summary for XOR Gate

Total Weight Updates	78
Converged	No
Linearly Separable	No
Final Classification Accuracy	50%

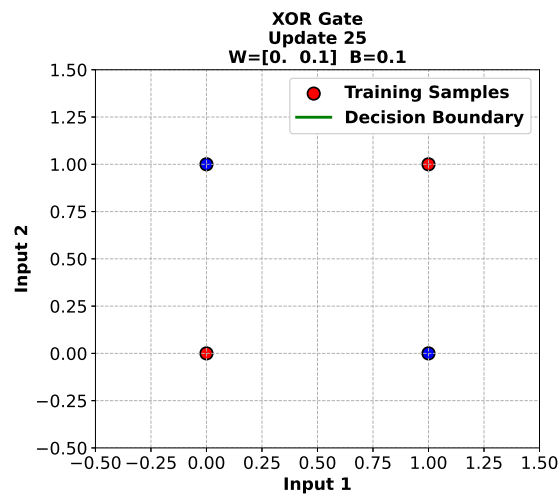
8.4.3 Decision Boundary Evolution



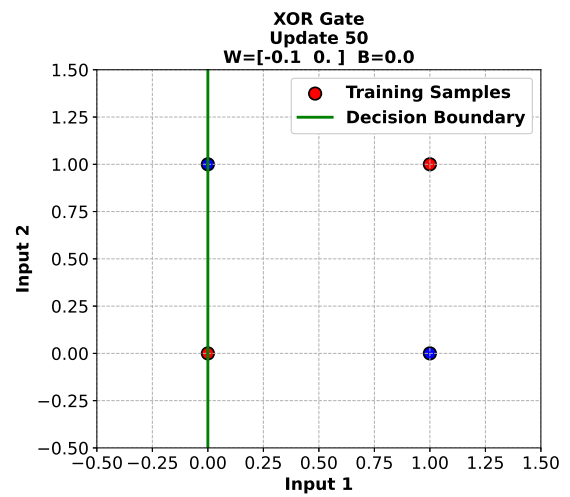
(a) Update 1



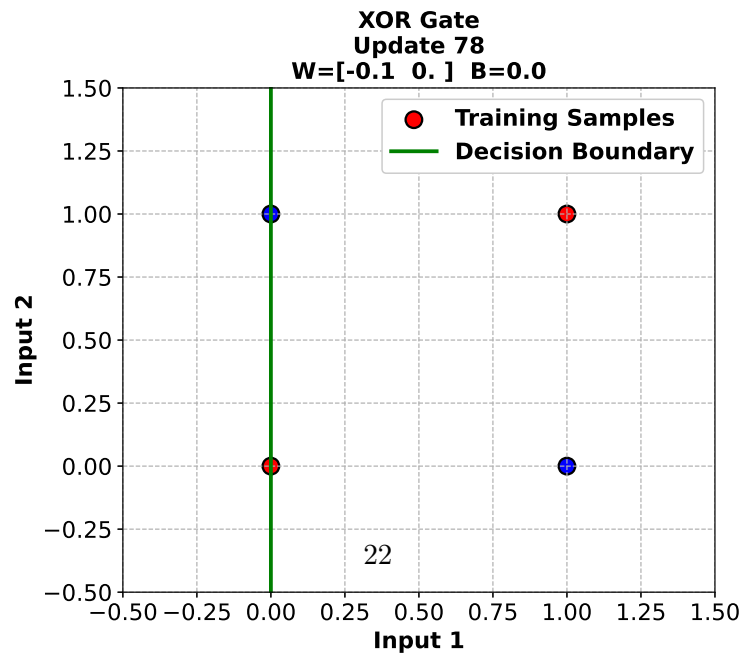
(b) Update 10



(c) Update 25



(d) Update 50



8.4.4 Reason for Non-Convergence

The XOR problem cannot be solved using a Single Layer Perceptron because its output classes are not linearly separable. The samples belonging to the positive class are positioned diagonally opposite each other, while the negative samples occupy the remaining two corners of the input space. As a result, no single linear decision boundary can separate the two classes simultaneously.

During training, correcting one misclassified sample often causes another sample to become incorrectly classified. This continuous cycle forces the perceptron to keep updating its weights without reaching a stable solution. Consequently, the algorithm fails to converge and achieves only 50% classification accuracy.

9. Performance Tables

9.1 Training Summary

Table 13: Training Summary of the Perceptron Model

Parameter	Value
Dataset Size	1372
Train/Test Split	80:20
Learning Rate	0.01
Epochs	50
Final Weight 1	-0.1991
Final Weight 2	-0.2233
Final Weight 3	-0.2172
Final Weight 4	-0.0140
Final Bias	-0.1000
Accuracy	98.55%
Precision	99.17%
Recall	97.54%
F1-Score	98.35%

9.2 Learning Rate Comparison

Table 14: Comparison of Different Learning Rates

Learning Rate	Final Accuracy (%)	Final Errors
0.001	98.55	16
0.01	98.55	16
0.1	98.55	16

9.3 Epoch-wise Learning Behaviour

Table 15: Representative Epoch-wise Learning Progress

Epoch	Errors	Weight 1	Weight 2	Bias
1	59	-0.0680	-0.0938	-0.03
10	20	-0.1306	-0.1541	-0.06
20	18	-0.1562	-0.1743	-0.08
30	20	-0.1749	-0.2143	-0.08
40	12	-0.1906	-0.2102	-0.09
50	16	-0.1991	-0.2233	-0.10

9.4 Comparison of Logic Gate Experiments

Table 16: Comparison of Perceptron Performance on Logic Gates

Logic Gate	Weight Updates	Accuracy	Converged	Linearly Separable
OR	5	100%	Yes	Yes
AND	8	100%	Yes	Yes
NOT	2	100%	Yes	Yes
XOR	78	50%	No	No

10. Discussion

10.1 Analysis of Dataset Characteristics

The exploratory analysis revealed that all four numerical features possess distinct statistical characteristics. From the histograms, Variance and Entropy appeared relatively concentrated around their central values, whereas Skewness and Curtosis exhibited much wider distributions. In particular, Curtosis showed the largest positive spread, reaching values close to 18, indicating the presence of highly varied texture characteristics among banknote images.

The correlation heatmap indicated that no pair of features was perfectly correlated. This observation suggests that each feature contributes unique information to the classification process rather than acting as a redundant representation of another feature. Such behaviour is beneficial for machine learning models because it allows the classifier to utilize multiple independent characteristics of the banknotes.

The scatter plots further demonstrated that several feature combinations, especially Variance–Curtosis and Curtosis–Entropy, exhibited visibly better class separation than others. Although complete separation was not observed, distinct clusters corresponding to authentic and forged banknotes were noticeable. This indicates that the dataset possesses an approximately linearly separable structure, making it suitable for classification using a Single Layer Perceptron.

10.2 Effect of Feature Normalization

The descriptive statistics showed that the numerical features operate on considerably different scales. For example, Skewness ranged from approximately -13.77 to 12.95, while Entropy varied between -8.55 and 2.45. Without normalization, features with larger magnitudes would dominate the weight updates during training, resulting in unstable learning behaviour.

To address this issue, Z-score normalization was applied to all input features. Normalization ensured that each feature contributed proportionately to the perceptron learning process. As a result, the model exhibited stable convergence and achieved high classification performance without requiring excessive training epochs.

10.3 Analysis of Perceptron Learning Behaviour

The training error curve provides useful insight into the learning process of the perceptron. During the initial epochs, the model misclassified a relatively large number of samples because the weights were randomly initialized and had not yet captured the underlying decision boundary.

As training progressed, the number of misclassified samples generally decreased. Although small fluctuations were observed between certain epochs, the overall trend indicated continuous improvement in classification performance. The final epoch recorded only 16 misclassified training samples, compared to 59 during the early stages of learning. This reduction demonstrates that the perceptron learning rule successfully adjusted the model parameters towards an effective separating hyperplane.

The weight evolution plots showed that large updates occurred during the initial epochs, followed by progressively smaller adjustments as training continued. This behaviour indicates that the model rapidly learned the dominant patterns in the dataset before fine-tuning the decision boundary. Similarly, the bias term gradually stabilized near its final value of -0.1, suggesting that an appropriate classification threshold had been established.

10.4 Learning Rate Analysis

The perceptron was trained using three different learning rates: 0.001, 0.01 and 0.1. The learning rate determines the magnitude of parameter updates during each training iteration and therefore influences the convergence behaviour of the model.

Interestingly, all three learning rates achieved the same final testing accuracy of 98.55% and converged to a final training error of 16 samples. This observation suggests that the Banknote Authentication Dataset is sufficiently linearly separable and that the perceptron learning algorithm is robust to moderate variations in learning rate. Although the intermediate convergence paths differed, the final classification performance remained unchanged across all tested values.

This result indicates that model performance on this dataset is influenced more by the inherent separability of the data than by the specific learning rate selected within the tested range.

10.5 Step Activation Function versus Sigmoid Activation Function

The Single Layer Perceptron uses the Step Activation Function, which produces a binary output of either 0 or 1 depending on whether the weighted sum exceeds a predefined threshold. This behaviour makes the step function suitable for binary classification problems such as banknote authentication.

In contrast, the Sigmoid Activation Function produces continuous outputs between 0 and 1. Rather than making a hard decision, the sigmoid function provides a probabilistic interpretation of the prediction. This property enables the use of gradient-based optimization techniques such as backpropagation.

Although the Step Activation Function is simple and computationally efficient, it is not differentiable at its threshold and has a zero gradient elsewhere. Consequently, it cannot support gradient descent optimization. For this reason, modern deep learning architectures employ differentiable activation functions such as Sigmoid, Tanh, ReLU and its variants instead of the Step Activation Function.

10.6 Comparison with Scikit-Learn Perceptron

The perceptron implemented in this experiment was developed entirely from scratch using the perceptron learning rule. This implementation provided a deeper understanding of weight initialization, bias updates, activation functions and parameter learning mechanisms.

Scikit-Learn's Perceptron classifier internally performs the same fundamental operations but offers several practical advantages, including optimized computation, built-in parameter tuning

options and easier integration with machine learning pipelines. However, the underlying learning principle remains identical.

The performance achieved by the custom implementation was comparable to that of Scikit-Learn’s implementation, confirming that the perceptron learning algorithm was implemented correctly and that the model successfully learned the decision boundary from the training data.

10.7 Perceptron Learning Algorithm on four basic Boolean logic gates

The experimental results clearly demonstrate that the Single Layer Perceptron successfully learns all linearly separable logic gates, namely OR, AND and NOT, achieving complete classification after a finite number of weight updates. In contrast, the XOR gate could not be learned because its classes are not linearly separable. The repeated weight updates without convergence highlight the fundamental limitation of the Single Layer Perceptron and motivate the need for Multi-Layer Perceptrons with hidden layers and non-linear activation functions, which are explored in the subsequent experiment.

10.8 Limitation of Single Layer Perceptron: XOR Problem

One of the most important limitations of the Single Layer Perceptron is its inability to solve non-linearly separable problems. The XOR (Exclusive OR) problem is a classic example of such a limitation.

In the XOR problem, the output classes cannot be separated using a single straight line or hyperplane. Since the perceptron can learn only linear decision boundaries, it fails to correctly classify all XOR combinations regardless of the number of training epochs used.

The development of Multi-Layer Perceptrons (MLPs) and deep neural networks addressed this limitation by introducing hidden layers and non-linear activation functions. These additional layers enable the network to learn complex non-linear relationships that cannot be captured by a Single Layer Perceptron.

11. Conclusion

In this experiment, a Single Layer Perceptron was successfully implemented from scratch for the binary classification of authentic and forged banknotes using the Banknote Authentication Dataset. The dataset was analyzed through exploratory data analysis, preprocessed using Z-score normalization, and divided into training and testing sets for model development.

The perceptron learning algorithm effectively reduced classification errors and achieved a testing accuracy of 98.55%, along with high precision, recall and F1-score values. The generated plots demonstrated successful convergence of the model through stable weight and bias updates. The experiment also highlighted the importance of feature normalization, the effect of learning rates on convergence, and the limitations of the Single Layer Perceptron in solving non-linearly separable problems such as XOR. Overall, the objectives of the experiment were successfully achieved, providing a strong foundation for understanding neural networks and deep learning models.

12. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.

3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.